Face Attendance System with Role-Based Access

This presentation outlines the architecture and functionality of a face attendance system, enhanced with role-based access control. This system leverages modern technologies to provide a secure and efficient method for tracking attendance, while also providing a flexible user interface with restricted access based on user roles.

This system combines facial recognition with role-based permissions to restrict access to key features. In addition to enhancing security, role-based access helps streamline system operations and prevent unauthorized access to sensitive functions. This presentation will provide a technical overview of its implementation, covering both the frontend and backend components.

by Priyesh Gupta

Made with GAMMA

Project Objective: Secure Facial Recognition Attendance

Efficient Attendance Tracking

Automate attendance marking using facial recognition technology.

Secure Access Control

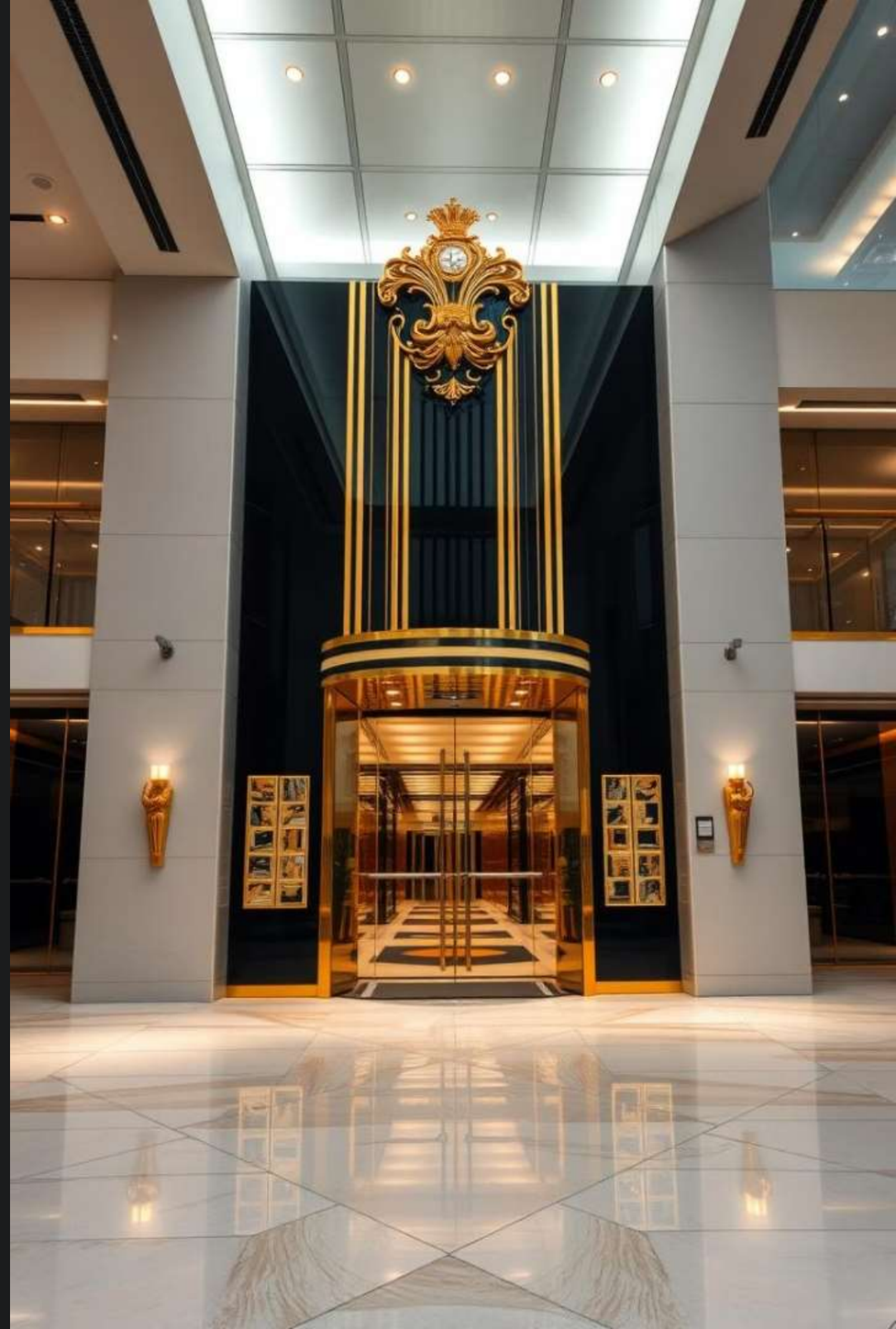
Implement role-based access to protect sensitive features.

Admin Interface

Enable user registration and attendance record management.

The primary objective is to develop a system that accurately recognizes faces for marking attendance and restricts access to sensitive functions based on user roles. The system's efficiency will be ensured via automated attendance tracking using facial recognition technology and a role-based access control mechanism.

The admin interface will streamline user registration, allowing administrators to easily manage user accounts and ensure that only authorized personnel can access administrative features. This will include viewing and managing attendance records and registering other administrators.



Key Features: Roles and Permissions

Authentication & Roles

- Admin and User login system
- Role stored in database
- Features restricted by role

Admin Features

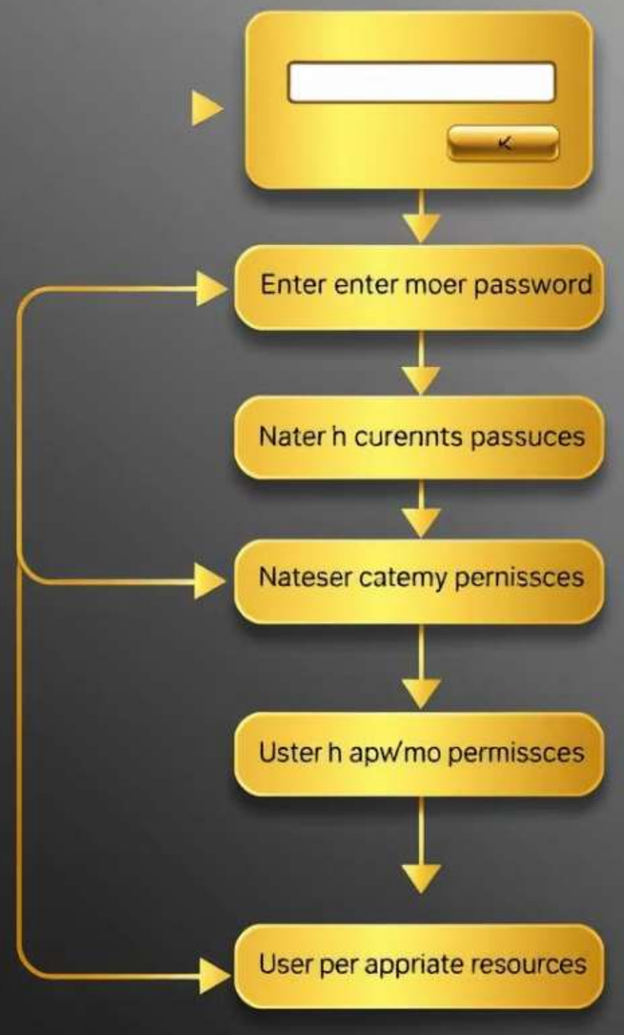
- Upload Face (register users)
- View Attendance (table records)
- Register another Admin

User Features

- Mark Attendance (face match)

The system features a dual-role setup with distinct functionalities for admins and regular users. Authentication is managed through a login system that stores roles in the database, enforcing access control throughout the application. Access to key features is restricted based on role. Admins can register users and view attendance records while users can only mark their attendance.

Made with GAMMA



Login Flow: Verification and Persistence

1

User Login

Users enter credentials on the login page.

2

Credential Verification

Flask API checks against hashed passwords.

3

Role Storage

Role and user info stored in localStorage.

4

Dynamic Navigation

Navbar options based on user role.

The login process ensures user credentials are securely verified via the Flask API, cross-referencing against hashed passwords in the database. On successful login, the user's role and information are stored in the browser's localStorage, maintaining session context and enabling role-specific navigation. The navigation bar dynamically renders links based on the user's role, ensuring that only appropriate options are visible.

Made with GAMMA

Face Recognition Workflow

Image Upload

User uploads a face image.

Encoding Comparison

Backend compares against stored face encodings.

Attendance Record

Attendance recorded in the database with a timestamp.

Frontend Result

Result returned (e.g., "Welcome back, John!")

The face recognition workflow begins with the user uploading an image of their face. The backend then compares this image against stored face encodings to find a match. If a match is found, the system records the attendance in the SQL database with a timestamp, indicating the exact time of entry. The frontend receives a result message confirming the successful attendance mark, such as "Welcome back, John!"

Made with GAMMA

ID	Shans	recun	TU:	TIDACD	TIMels
13	• L.1L	10050	020	39900	23000
14	• L.1L	10050	020	35900	23000
15	• L.1L	15000	020	32000	22000
19	• 2.1CBlejs	10000	000	32500	17800
15	• 2.1CBlejs	10000	000	23900	27600
15	• 2.1CElejs	10000	000	33000	11000
16	• L.1CE	15060	000	35000	17800
17	• 6.1CElelorse	10000	000	33500	11800
10	• 6.1CElerats	10000	000	33600	11800
19	• 6 1CBle	10030	000	33900	17800
18	• 2.1CEleys	10000	000	32500	17800
14	• 2.1CElerats	10050	000	13560	11890
16	• 2.1CEl	15000	060	33500	11800
19	• 2.1CEle	10000	000	35500	11800
17	• 2.1L	15050	000	33900	17900

Attendance Tracking with SQL Database

Field	Description
id	Unique identifier for each record.
username	The username of the person.
timestamp	The date and time when attendance was marked.

Attendance data is stored in a SQL database, designed to log each attendance record efficiently. Each record includes a unique ID, the username, and the timestamp, providing a comprehensive audit trail of attendance activity. The database is structured to ensure data integrity and facilitate quick retrieval for reporting and analysis.

The data stored in the SQL database is used to populate the attendance table. This table is displayed via a React AttendanceTable component in the frontend, giving administrators a clear and organized view of all attendance records.

Made with GAMMA

Frontend Route Protection and UI Control

1

Navbar Rendering

Links rendered based on logged-in user's role.

2

Admin-Only Pages

UploadFace, RegisterAdmin, and Attendance.

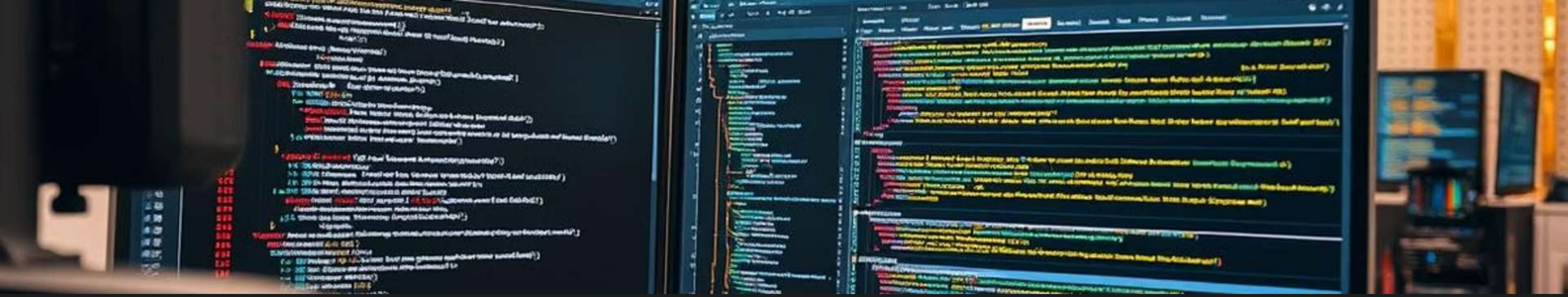
3

Shared Access

MarkAttendance accessible to both roles.

The frontend ensures secure access control by dynamically rendering navigation links based on the logged-in user's role. Pages such as UploadFace, RegisterAdmin, and Attendance are exclusively accessible to administrators, ensuring that sensitive functions are protected. MarkAttendance is accessible to both roles, providing all users with the ability to mark their attendance seamlessly.

Made with GAMMA



Rigorous Testing and Validation

API Testing

Verified API functionality in Postman.

Frontend Testing

Login/attendance tested via frontend UI.

Error Handling

Handled invalid credentials and no face detected errors.

The system has undergone thorough testing to ensure its functionality and reliability. API endpoints were tested using Postman to verify that all functions are working as expected. Additionally, the login and attendance features were tested via the frontend UI to simulate user interactions. Comprehensive error handling was implemented to manage scenarios such as invalid credentials and cases where no face is detected in the uploaded image.

Made with GAMMA

Future Improvements for Enhanced Scalability

Real-time Capture

Implement camera capture for attendance.

JWT Auth

Enhance security with JWT-based authentication.

Auto Logout

Implement session expiration.

Analytics Dashboard

Visualize attendance data.

Future improvements could include integrating real-time camera capture for marking attendance and enhancing security with JWT-based authentication. Auto logout or session expiration can be added to improve security. Introducing an attendance analytics dashboard would provide a clear overview of attendance trends. Also, the system can be deployed to a cloud platform like Render for the Flask backend and Vercel for the React frontend, enhancing scalability and availability.

Made with GAMMA

GITHUB LINK: https://github.com/krishan04/FaceAttendanceSystem

Made with GAMMA